



2026

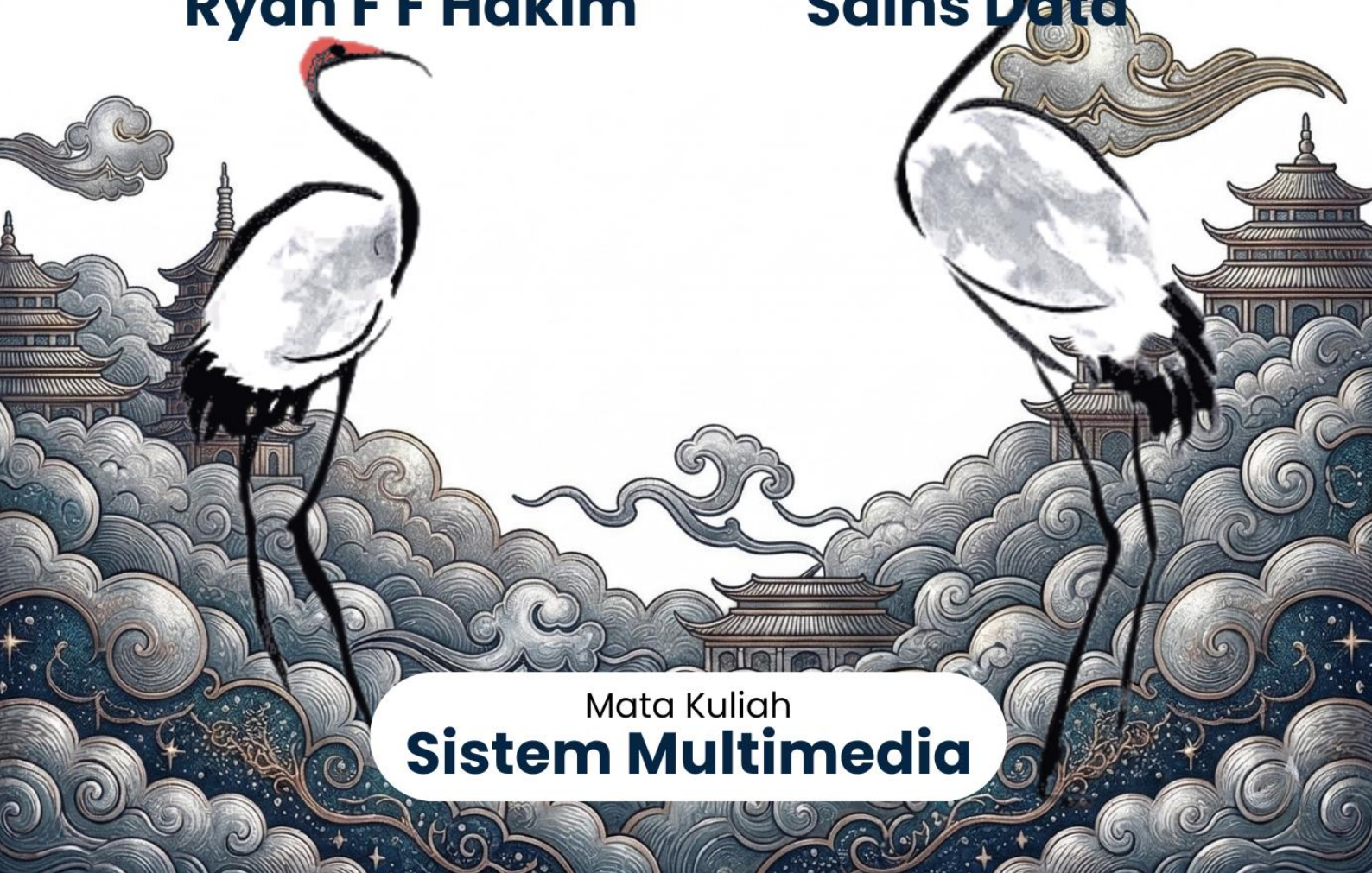
MODUL PRAKTIKUM 3
BASIC DIGITAL IMAGE PROCESSING
OPTIMIZED VERSION

Disusun Oleh:

Ryan F F Hakim

Dipersembahkan Untuk:

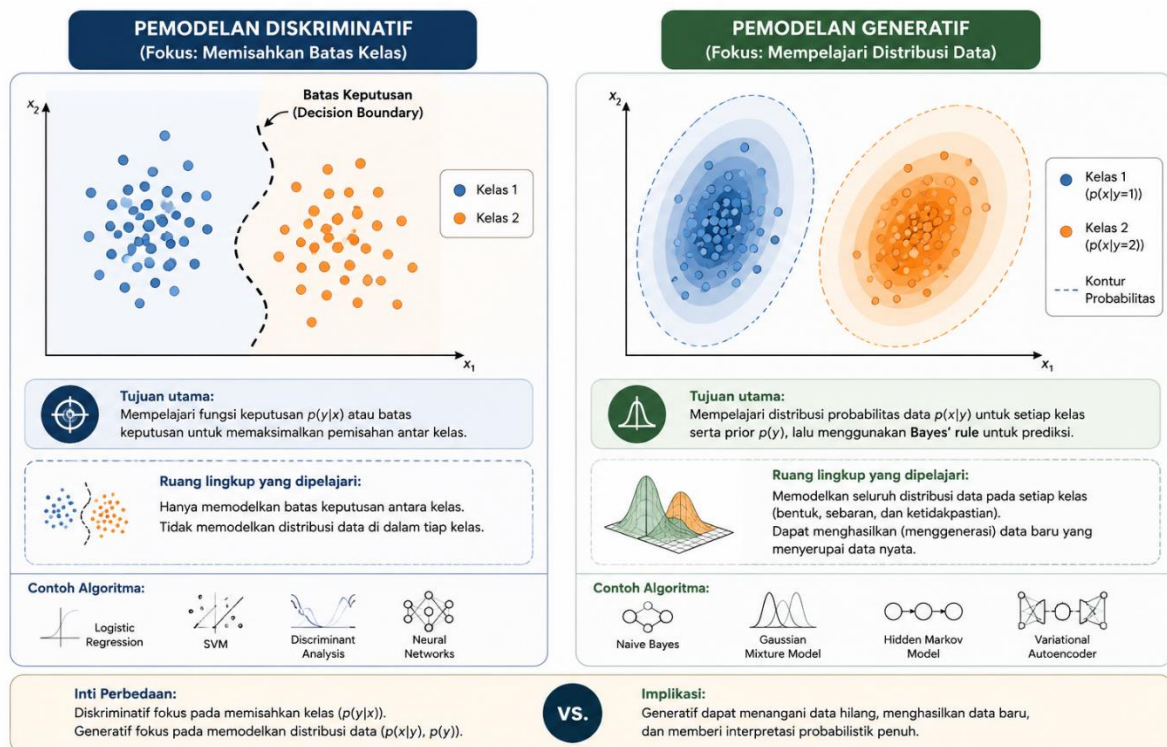
Sains Data



Mata Kuliah
Sistem Multimedia

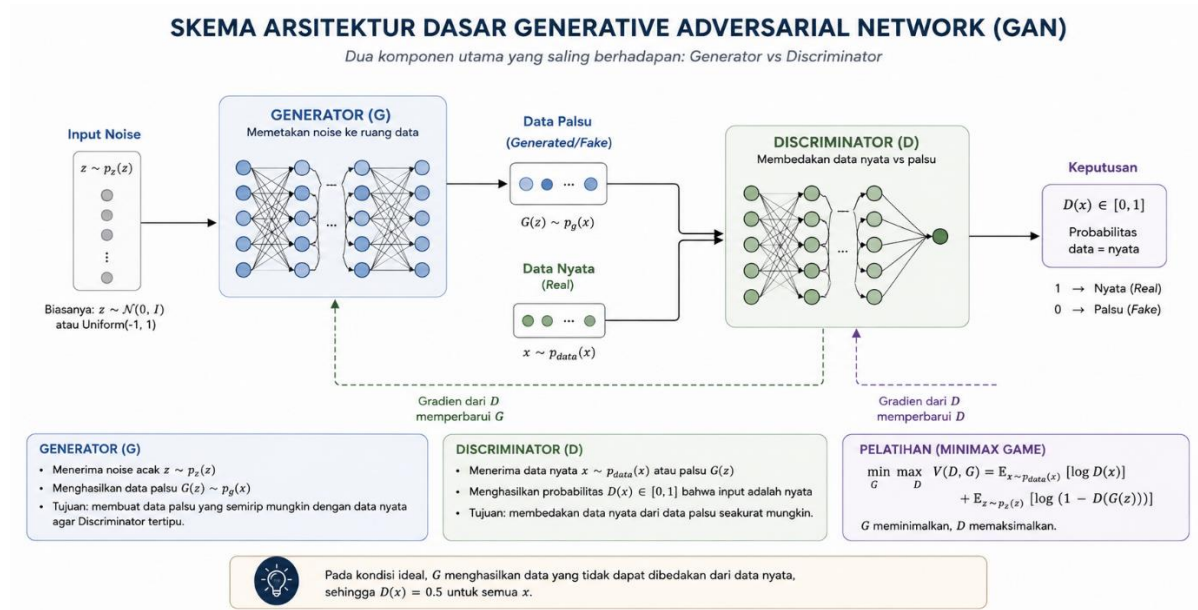
PENDAHULUAN

Perkembangan kecerdasan buatan, khususnya pada bidang *deep learning*, telah mengalami kemajuan pesat dalam beberapa dekade terakhir. Fokus utama pembelajaran mesin kini tidak lagi hanya terbatas pada tugas diskriminatif seperti klasifikasi atau pengenalan pola, melainkan telah meluas pada kemampuan komputasi untuk menghasilkan data baru yang menyerupai data asli. Pendekatan ini secara formal dikenal sebagai model generatif. Melalui pemodelan generatif, distribusi probabilitas dari himpunan data nyata dipelajari, sehingga sintesis sampel data baru yang realistis dapat dilakukan secara otomatis.



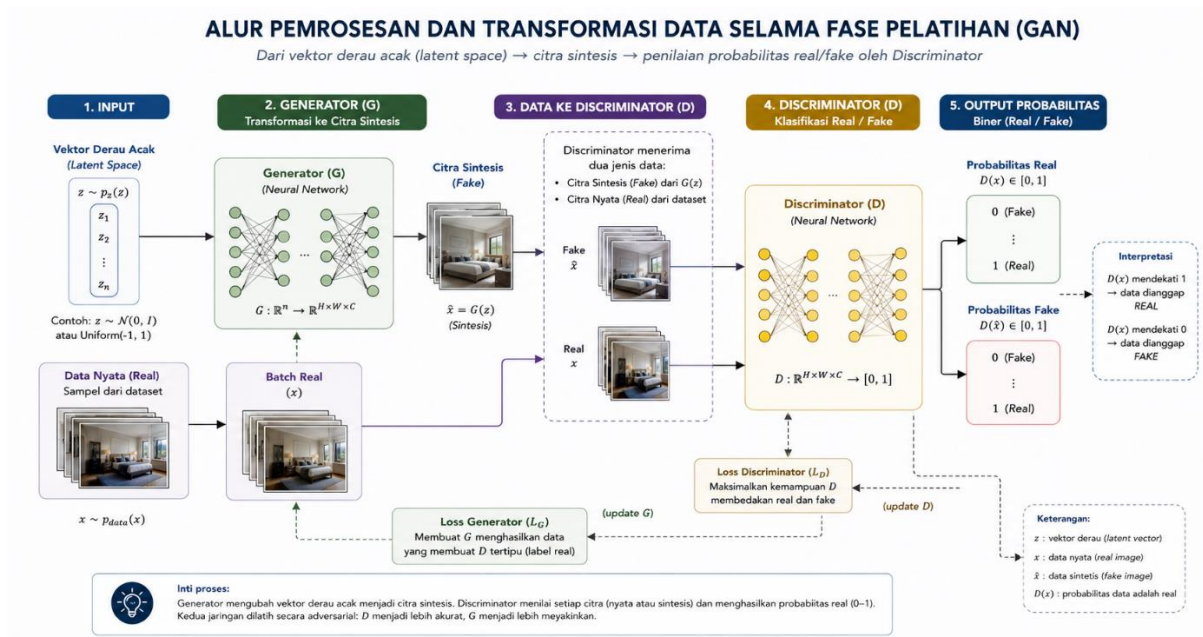
Gambar 1.1: Ilustrasi konseptual yang menampilkan perbandingan antara ruang lingkup pemodelan diskriminatif dalam memisahkan batas kelas data, dan pemodelan generatif dalam mempelajari serta mereplikasi distribusi probabilitas data nyata.

Salah satu arsitektur model generatif yang paling menonjol dan revolusioner adalah *Generative Adversarial Network* (GAN). Diperkenalkan pertama kali oleh Ian Goodfellow dan rekan-rekannya pada tahun 2014, GAN beroperasi berdasarkan konsep persaingan (*adversarial*) antara dua jaringan saraf tiruan independen, yang masing-masing disebut sebagai *Generator* dan *Discriminator*. Pendekatan struktural ini memungkinkan sistem untuk mempelajari representasi data kompleks tanpa memerlukan fungsi objektif spesifik yang dirumuskan secara manual.



Gambar 1.2: Skema arsitektur dasar Generative Adversarial Network (GAN) yang menunjukkan dua komponen utama berupa jaringan Generator dan jaringan Discriminator yang saling berhadapan.

Mekanisme operasional GAN secara mendasar dibangun di atas prinsip permainan *minimax* dari teori permainan (*game theory*). Dalam struktur ini, *Generator* ditugaskan untuk memetakan vektor derau acak (*random noise*) menjadi data tiruan dengan tujuan utama mengelabui jaringan lawannya. Sebaliknya, *Discriminator* dilatih sebagai pengklasifikasi biner untuk membedakan secara akurat mana data sampel yang berasal dari distribusi data nyata dan mana data buatan hasil sintesis *Generator*. Proses pelatihan kedua jaringan ini dieksekusi secara bersamaan dan berkesinambungan hingga tercapai suatu titik keseimbangan (*Nash Equilibrium*), yaitu saat probabilitas *Discriminator* dalam mengenali data asli maupun palsu berada pada tingkat acak yang sama karena tingginya kualitas data tiruan yang diproduksi.



Gambar 1.3: Alur pemrosesan dan transformasi data selama fase pelatihan, mendemonstrasikan bagaimana vektor derau acak (latent space) diubah menjadi citra sintesis, serta proses penentuan probabilitas biner (real/fake) oleh Discriminator terhadap data masukan.

Modul praktikum ini disusun untuk menjembatani pemahaman konseptual teoretis dengan keterampilan implementasi praktis di bidang pemodelan generatif. Di dalam tahapan praktikum, prosedur perancangan arsitektur dasar *Generator* dan *Discriminator* akan diuraikan langkah demi langkah menggunakan *framework deep learning* modern. Selain itu, alur pelatihan jaringan yang melibatkan perhitungan fungsi kerugian (*loss function*) spesifik GAN, mekanisme pembaruan bobot (*backpropagation*), serta visualisasi hasil pembangkitan data seiring berjalannya iterasi (*epoch*) akan dikaji secara mendalam.



Gambar 1.4: Visualisasi komparatif perkembangan kualitas citra hasil sintesis dari Generator, yang direkam secara periodik dari iterasi pelatihan (epoch) awal yang masih berupa derau hingga iterasi akhir yang membentuk pola realistis.

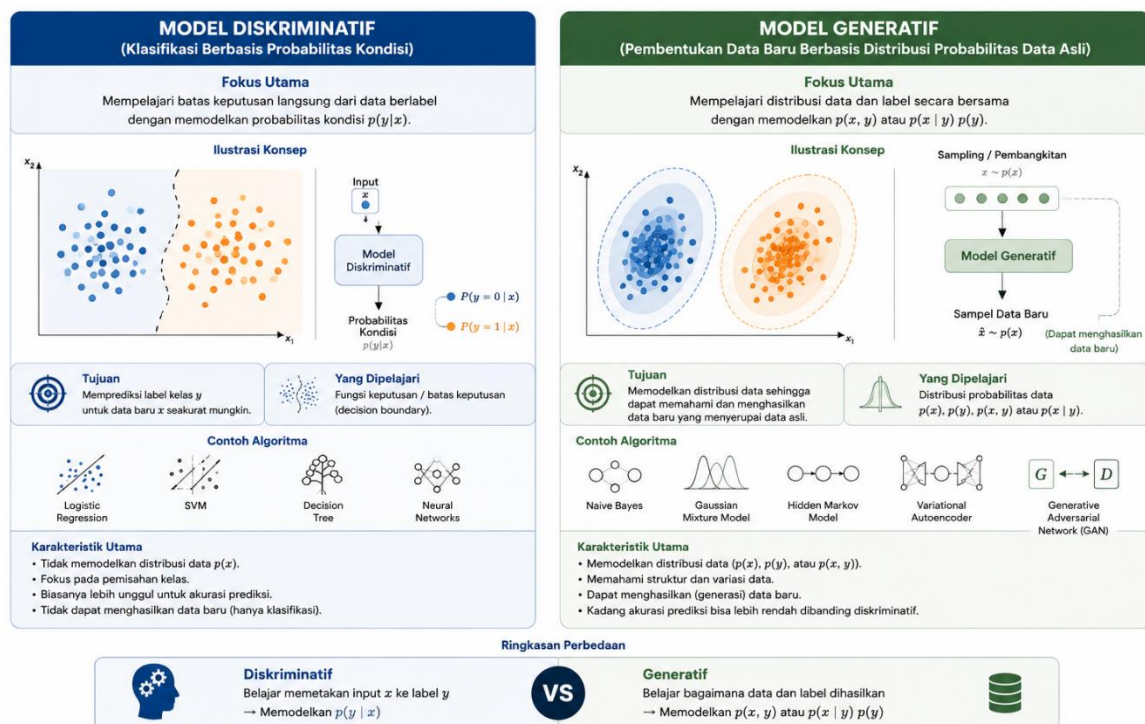
Penyelesaian seluruh tahapan eksperimen di dalam modul ini diharapkan dapat membekali mahasiswa dengan kemampuan analitis dan teknis dalam merancang, melatih, serta mengevaluasi model GAN secara mandiri. Pemahaman komprehensif terhadap dinamika pelatihan *adversarial* sangat esensial, mengingat berbagai tantangan teknis seperti ketidakstabilan konvergensi dan hilangnya keberagaman sampel (*mode collapse*) sering menjadi kendala di lapangan. Melalui penguasaan materi fundamental ini, fondasi yang kokoh dapat dibangun sebagai landasan eksplorasi terhadap varian arsitektur GAN yang lebih mutakhir di masa depan.

LANDASAN TEORI

2.1 PEMODELAN GENERATIF VS PEMODELAN DISKRIMINATIF

Dalam ranah *deep learning*, arsitektur seperti Artificial Neural Network (ANN), Convolutional Neural Network (CNN), dan Recurrent Neural Network (RNN) secara umum bersifat **diskriminatif**. Jaringan ini dirancang untuk mempelajari batas keputusan (*decision boundary*) guna membedakan atau mengklasifikasikan data (misalnya membedakan gambar kucing dan anjing).

Sebaliknya, **Generative Adversarial Network (GAN)** yang diperkenalkan oleh Ian Goodfellow et al. (2014) merupakan kelas arsitektur yang bersifat **generatif**. Alih-alih hanya membedakan data, model ini mampu mempelajari distribusi dari data asli dan **menciptakan data baru** (sintetis) yang belum pernah ada sebelumnya namun terlihat sangat nyata dan realistis.



Gambar 2.1: Perbedaan mendasar model diskriminatif (klasifikasi berbasis probabilitas kondisi) dan model generatif (pembentukan data baru berbasis distribusi probabilitas data asli).

Perbandingan Karakteristik Arsitektur

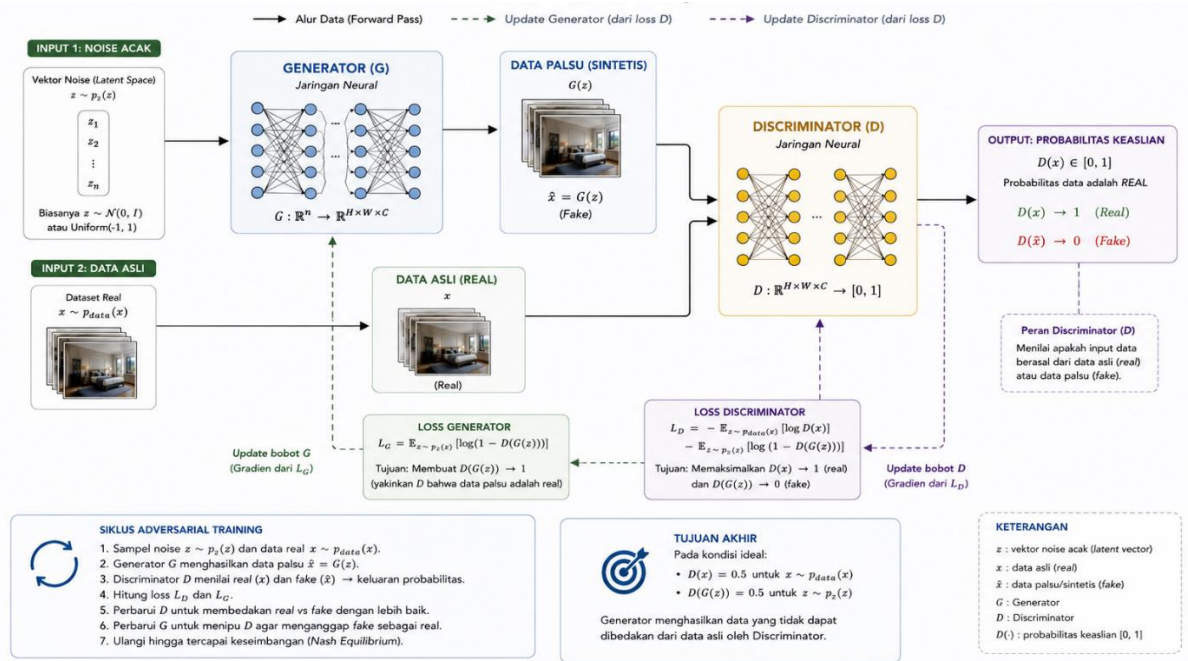
Aspek	Model Diskriminatif (ANN, CNN, RNN)	Model Generatif (GAN)
Fokus Utama	Mempelajari $P(y x)$ (probabilitas label y diberikan input x)	Mempelajari $P(x)$ atau $P(x, y)$ (distribusi probabilitas data x)
Bentuk Masukan	Matriks data fitur nyata (X)	Vektor <i>noise</i> acak (z)
Output Jaringan	Kelas, label, probabilitas, atau nilai regresi (y)	Sampel data baru atau sintetik (X')
Tujuan Akhir	Membedakan dan mengelompokkan data secara akurat	Menciptakan sampel data yang tidak bisa dibedakan dari data asli

2.2 ARSITEKTUR DASAR GENERATIVE ADVERSARIAL NETWORK (GAN)

Arsitektur GAN pada dasarnya tidak menggunakan sebuah model tunggal, melainkan tersusun atas dua model *neural network* terpisah yang dilatih secara bersamaan melalui proses yang disebut **adversarial training** (pelatihan dengan kompetisi/permusuhan). Kedua komponen tersebut adalah:

1. **Generator (G)**: Bertindak layaknya seorang "pemalsu". Jaringan ini menerima vektor acak (*noise*) berdimensi rendah sebagai masukan dan berusaha mengonversinya menjadi matriks data sintetik palsu yang menduplikasi ciri-ciri distribusi data asli.
2. **Discriminator (D)**: Bertindak sebagai seorang "detektif". Jaringan ini menerima masukan berupa data asli dari dataset maupun data palsu dari output Generator, kemudian bertugas mengukur probabilitas keaslian data tersebut.

Keduanya bersaing dalam sebuah siklus umpan balik: Generator secara konstan menyempurnakan kemampuannya untuk memproduksi manipulasi yang menipu Discriminator, sedangkan Discriminator secara bertahap menajamkan instingnya untuk mendeteksi data palsu. Kompetisi dinamis ini berlangsung hingga mencapai titik konvergensi *Nash Equilibrium*, yaitu keadaan di mana Discriminator tidak lagi dapat membedakan data asli dan palsu (probabilitas mencapai ambang 0.5 secara konsisten).



Gambar 2.2: Diagram blok arsitektur dasar Generative Adversarial Network (GAN) dalam siklus adversarial training. Skema ini memperlihatkan bagaimana jaringan Generator memproses vektor noise acak menjadi data palsu (sintetis), yang kemudian diumpankan ke jaringan Discriminator bersama-sama dengan data asli dari dataset. Discriminator bertindak sebagai penilai untuk mengukur probabilitas keaslian data, yang hasilnya menjadi basis evaluasi bagi kedua jaringan untuk saling menyempurnakan performanya.

2.3 FORMULASI MATEMATIS FUNGSI OBJEKTIF (ADVERSARIAL LOSS)

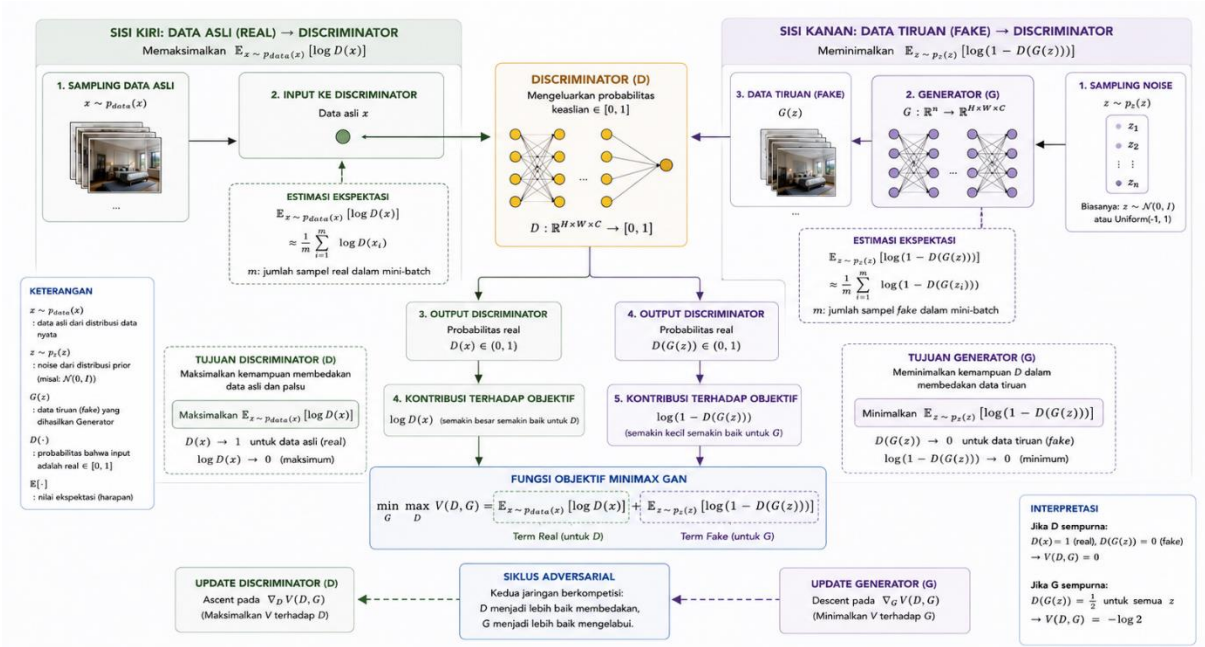
Interaksi kompetitif pada GAN secara matematis dimodelkan menggunakan kerangka teoritis permainan *minimax* (Minimax Game). Fungsi objektif (*value function*) $V(D, G)$ diformulasikan ke dalam persamaan logaritma berikut:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Keterangan parameter matematis:

- x : Sampel data asli yang ditarik secara acak dari distribusi empiris data nyata $p_{data}(x)$.
- z : Vektor *noise* acak yang diekstraksi dari ruang laten (berbasis distribusi normal multivariat standar, $p_z(z)$).
- $G(z)$: Entitas data palsu sintetis yang di-generate melalui jaringan Generator.
- $D(x)$: Probabilitas prediktif dari Discriminator yang menunjukkan bahwa input x adalah data riil asli.

- $D(G(z))$: Probabilitas prediktif dari Discriminator yang menunjukkan bahwa input tiruan $G(z)$ adalah data riil asli.
- $\mathbb{E}$: Nilai ekspektasi matematis rata-rata (*expected value*).

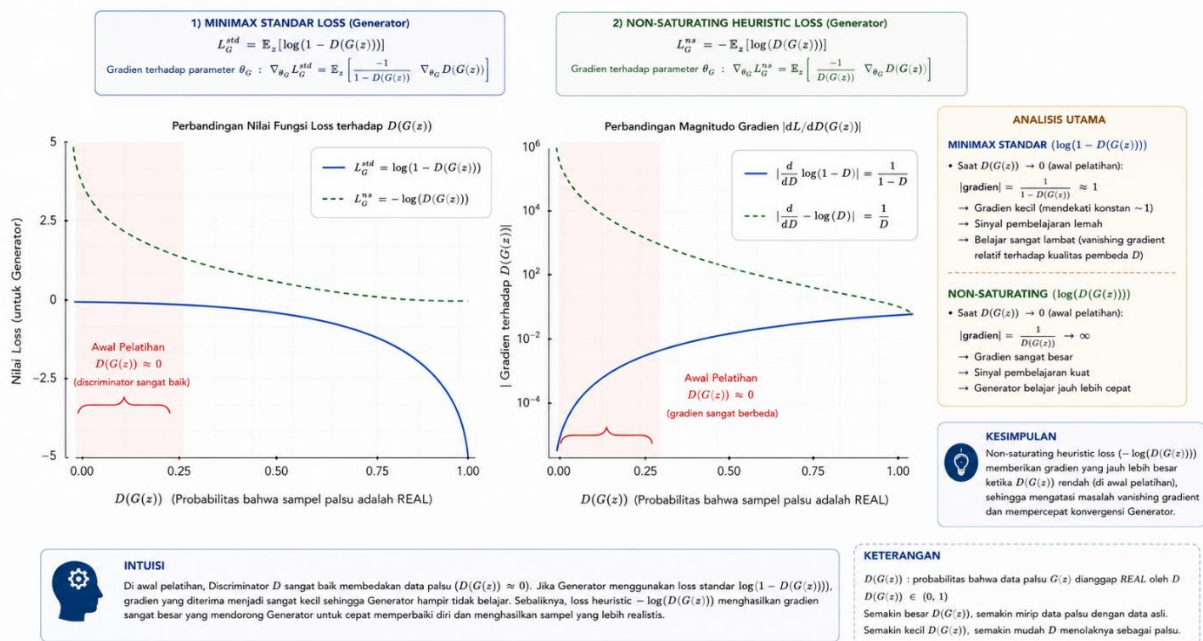


Gambar 2.3: Ilustrasi alur data dan pemetaan parameter fungsi objektif minimax pada arsitektur GAN. Sisi kiri menunjukkan proses kalkulasi nilai ekspektasi data asli $\mathbb{E}_{x \sim p_{data}(x)} [\log D(x)]$ di mana Discriminator (D) berusaha memaksimalkan akurasi prediksinya. Sisi kanan menunjukkan alur Generator (G) yang mengubah noise (z) menjadi data tiruan $G(z)$ untuk meminimalkan nilai $\log(1 - D(G(z)))$ demi mengelabui Discriminator.

Penjabaran Paradigma Fungsi Objektif:

- **Perspektif Discriminator ($\max_D$):** Model Discriminator berambisi untuk **memaksimalkan** akurasi pendeteksiannya. Ia berusaha agar nilai parameter $D(x)$ mendekati asimtot 1 (mengidentifikasi data asli dengan benar) dan nilai parameter $D(G(z))$ mendekati 0 (mengidentifikasi data palsu dengan benar). Ini memaksimalkan keseluruhan nilai ekuasi $[\log D(x) + \log(1 - D(G(z)))]$.
- **Perspektif Generator ($\min_G$):** Model Generator menargetkan untuk **meminimalkan** akurasi keandalan Discriminator. Generator berusaha mengecoh probabilitas Discriminator agar $D(G(z))$ mendekati 1. Kalkulasi matematis ini ekuivalen dengan meminimalkan suku $\log(1 - D(G(z)))$.

Catatan Optimasi: Pendekatan meminimalkan $\log(1 - D(G(z)))$ seringkali memicu kendala lenyapnya gradien (*vanishing gradient*) pada awal pelatihan. Standar *deep learning* modern memodifikasi fungsi objektif Generator agar secara langsung **memaksimalkan** suku fungsi $\log(D(G(z)))$, dikenal sebagai *Non-saturating heuristic loss*.



Gambar 2.4: Grafik perbandingan gradien antara fungsi minimax loss standar $\log(1 - D(G(z)))$ dan non-saturating heuristic loss $\log(D(G(z)))$ untuk Generator. Kurva memperlihatkan bagaimana fungsi standar mengalami kelaian (gradien mendekati nol) saat nilai $D(G(z))$ rendah di awal pelatihan (*vanishing gradient*), sementara fungsi heuristic menyediakan gradien yang jauh lebih curam dan kuat untuk mempercepat proses pembelajaran Generator.

2.4 DINAMIKA SIKLUS PELATIHAN GAN

Berbeda halnya dengan topologi jaringan konvensional tunggal yang memperbarui bobot parameter menggunakan satu *loss function*, arsitektur GAN mensyaratkan mekanisme pelatihan ganda terpisah yang dieksekusi secara bergantian (*alternating optimization*).

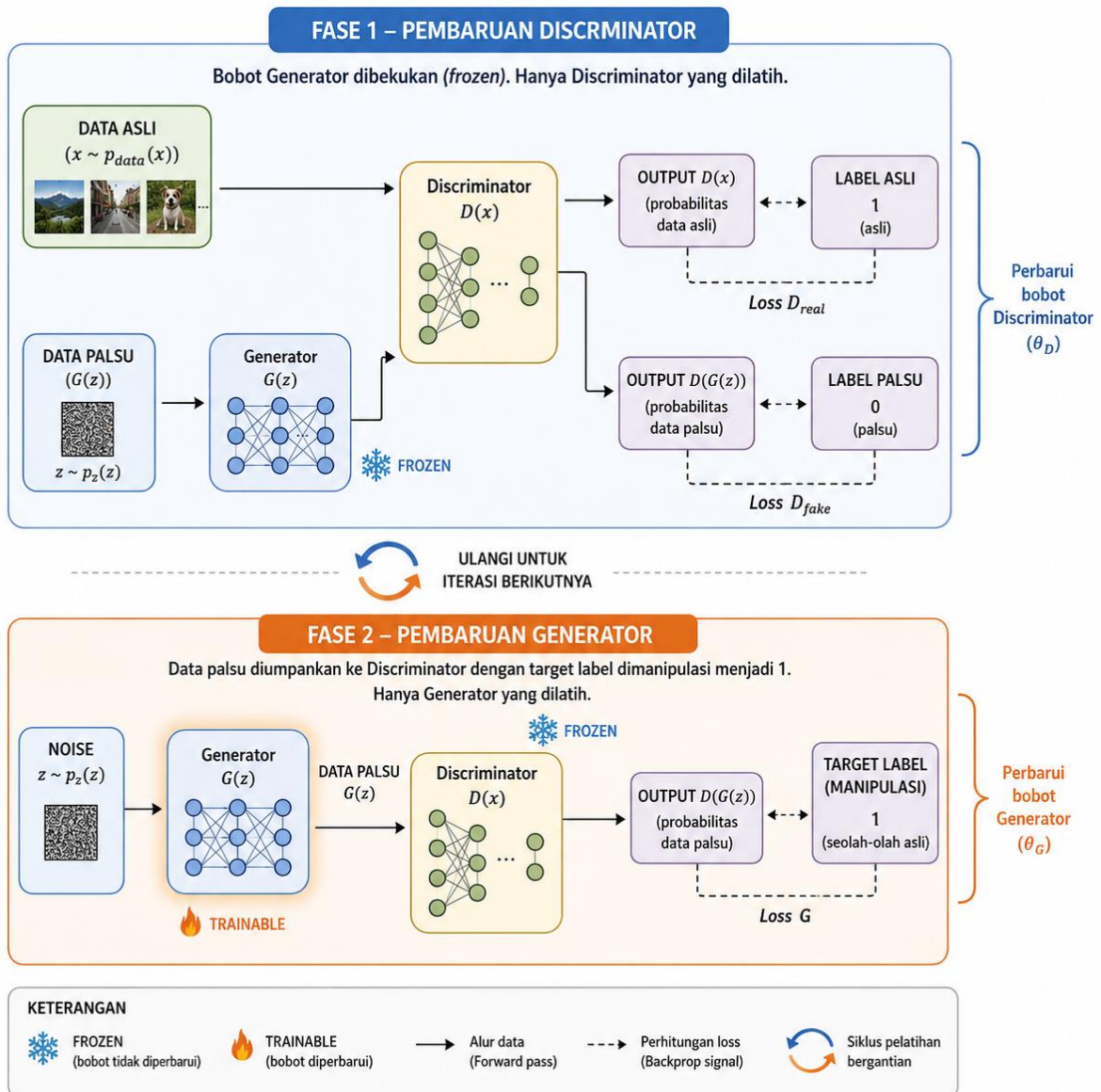
Langkah pelatihan jaringan dalam siklus satu iterasi (*batch training*) memecah operasi ke dalam dua fase independen:

1. Fase Pembaruan Discriminator:

- Melakukan inferensi pada representasi *batch* data **asli**. Loss dihitung menggunakan landasan metrik *Binary Cross Entropy* (BCE) dengan target label konstan = 1.
- Melakukan inferensi terhadap *batch* sampel data **palsu** manifestasi Generator. Loss dihitung kembali melalui BCE dengan target label = 0.
- Mengakumulasi nilai *total loss* dan melakukan propagasi alur mundur (*backpropagation*) **hanya** pada bobot model Discriminator.

2. Fase Pembaruan Generator:

- Jaringan Generator dieksekusi untuk memformulasikan *batch* data **palsu** fiktif yang baru.
- Data fiktif ini diteruskan ke lapisan Discriminator sebagai variabel tes penipuan.
- Loss dikalkulasi menggunakan turunan parameter BCE dengan memanipulasi target label = 1, merepresentasikan harapan Generator agar Discriminator mendiagnosis sampelnya sebagai data asli.
- Perputaran pembaruan bobot (*backpropagation*) diaplikasikan **hanya** pada parameter model Generator.



Gambar 2.5: Diagram alur siklus pelatihan bergantian (*alternating optimization*) dalam satu iterasi GAN. Fase atas menunjukkan proses pembaruan Discriminator di mana bobot Generator dibekukan (*frozen*) saat model melatih data asli (label 1) dan data palsu (label 0). Fase bawah menunjukkan proses pembaruan Generator di mana data palsu diumpankan ke Discriminator dengan target label yang dimanipulasi menjadi 1, dan pembaruan bobot hanya dilakukan pada jaringan Generator.

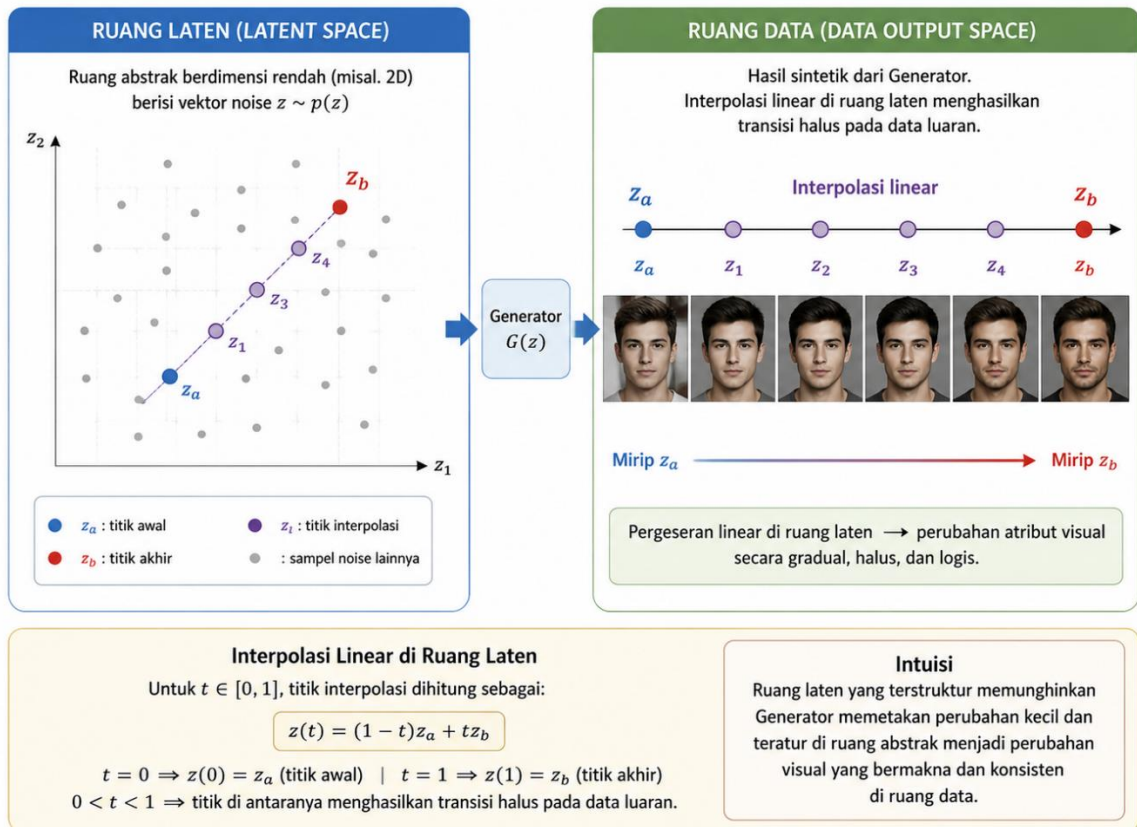
2.5 KONSEP LATENT SPACE (RUANG LATEN)

Input Generator bukanlah data terstruktur secara kognitif, melainkan vektor acak z yang berasal dari **Latent Space** (Ruang Laten). Latent space merupakan representasi matematis berdimensi rendah (*low-dimensional representations*) kompresi probabilitas yang merumuskan manifestasi model peta abstrak dari seluruh kemungkinan variasi dataset nyata.

Setiap koordinat titik dalam *latent space* berkorespondensi langsung pada karakteristik representatif yang eksklusif dari data sintetik hasil *generate*.

Properti Kunci Latent Space pada Area Domain GAN:

- **Dimensi Terkompresi Rendah (*Low-Dimensional Space*):** Vektor input acak direpresentasikan dalam dimensi yang jauh lebih kompak (umumnya berukuran 100 dimensi), yang secara signifikan lebih kecil jika dibandingkan dengan resolusi luaran akhirnya (seperti gambar 28x28 yang setara dengan 784 piksel).
- **Interpolasi Halus (*Smooth Interpolation*):** Perpindahan nilai koordinat yang berangsur dan linier di dalam ruang laten akan menghasilkan transisi perubahan yang halus pada data luaran sintetik. Sebagai contoh, pergeseran perlahan pada vektor ruang laten dapat secara gradual merubah atribut wujud gambar dari waktu ke waktu dengan proporsi perubahan yang logis.



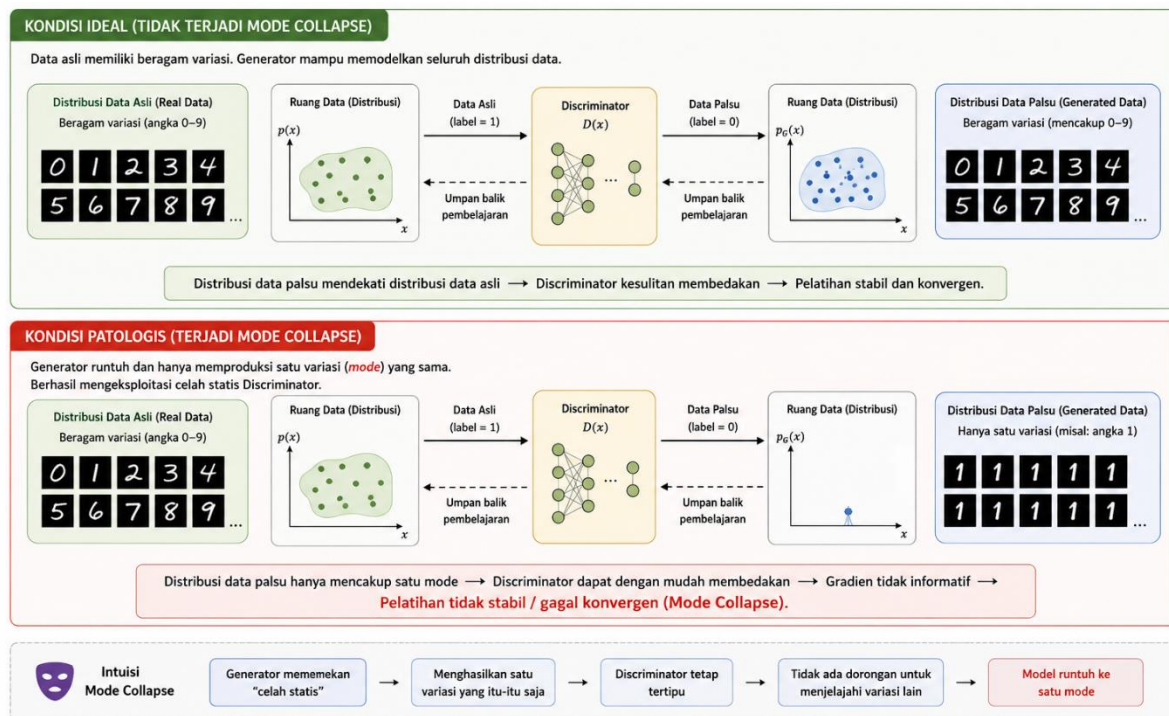
Gambar 2.6: Visualisasi konsep ruang laten (latent space) berdimensi rendah dan manifestasinya pada data luaran. Bagian kiri menunjukkan koordinat vektor noise acak (z) di dalam ruang abstrak, sementara bagian kanan memperlihatkan hasil transisi halus (smooth interpolation) pada gambar sintetik yang dihasilkan oleh Generator. Pergeseran linear antar-titik koordinat di ruang laten menghasilkan perubahan atribut visual data (misalnya bentuk atau karakteristik wajah) secara gradual dan logis.

2.6 TANTANGAN DAN KENDALA EKSLUSIF PADA ARSITEKTUR GAN

Pelatihan GAN yang bertumpu pada kompetisi *minimax* seringkali menginduksi ketidakstabilan numerik jika dibandingkan dengan arsitektur diskriminatif konvensional. Beberapa hambatan fungsional utamanya meliputi:

2.6.1. MODE COLLAPSE

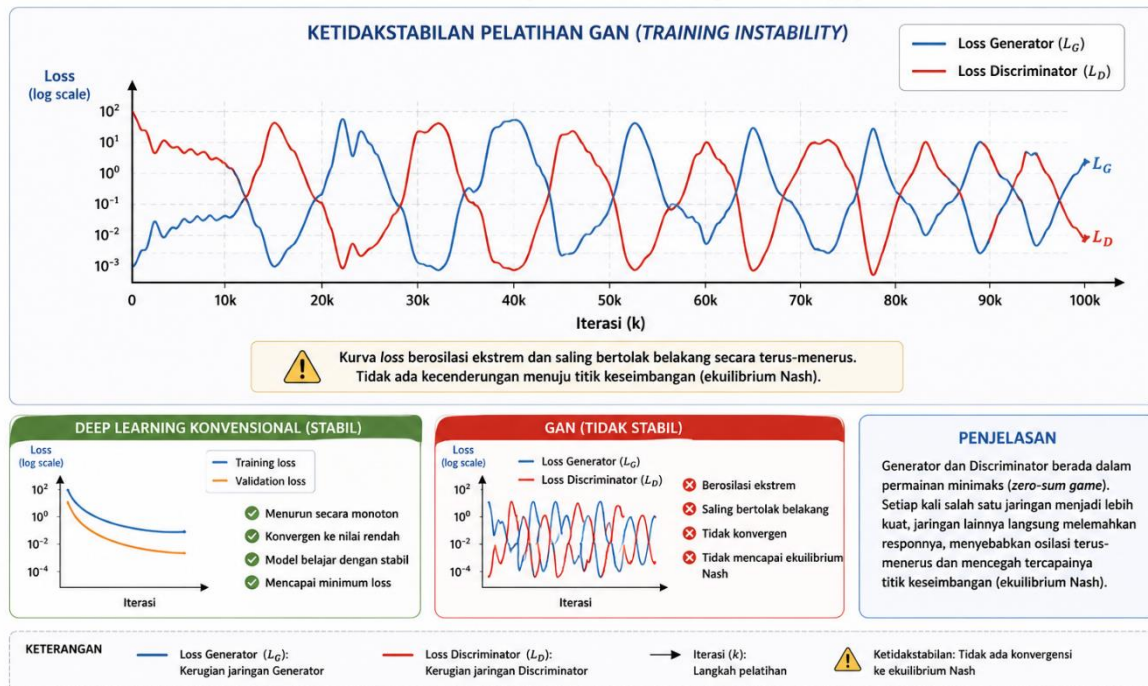
Kondisi patologis di mana unit Generator mengalami kegagalan dalam merepresentasikan seluruh distribusi keragaman pada data asli. Akibatnya, Generator "runtuh" dan hanya memproduksi satu atau sebagian kecil variasi bentuk (mode) berulang-ulang yang kebetulan berhasil menipu Discriminator. Discriminator terjebak karena tidak mampu memberikan penalti pada kurangnya keragaman ini.



Gambar 2.7: Ilustrasi fenomena ketidakstabilan Mode Collapse pada proses pelatihan GAN. Bagian atas menunjukkan distribusi data asli ideal yang memiliki beragam variasi (misalnya seluruh digit angka 0 sampai 9), sedangkan bagian bawah menunjukkan kondisi patologis di mana Generator runtuh dan hanya memproduksi satu variasi bentuk atau angka yang sama (misalnya angka 1 secara berulang-ulang) karena telah menemukan celah statis yang berhasil mengelabui Discriminator.

2.6.2. TRAINING INSTABILITY DAN NON-CONVERGENCE

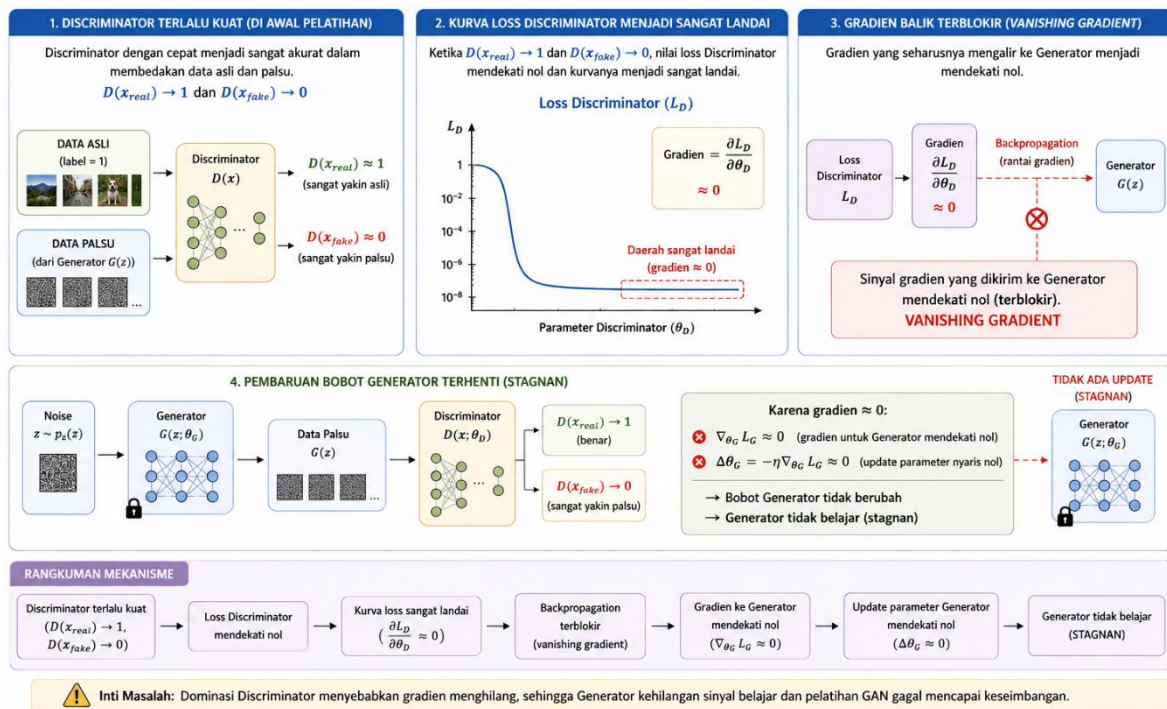
Permainan kompetitif antara Generator dan Discriminator rentan mengalami osilasi tanpa henti. Fluktuasi nilai *loss* seringkali bergerak ekstrem dan tidak menentu, sehingga model gagal untuk mencapai titik ekuilibrium atau konvergensi (pemusatan) di akhir proses pelatihan.



Gambar 2.8: Grafik pergerakan nilai fungsi kerugian (*loss curve*) yang menunjukkan fenomena ketidakstabilan pelatihan (*training instability*) pada GAN. Berbeda dengan kurva deep learning konvensional yang melandai secara monoton menuju konvergensi, kurva ini memperlihatkan osilasi ekstrem yang saling bertolak belakang secara terus-menerus antara *loss* Generator dan *loss* Discriminator, mengindikasikan kegagalan kedua jaringan dalam mencapai titik keseimbangan (*ekuilibrium Nash*).

2.6.3. VANISHING GRADIENT

Skenario di mana Discriminator menjadi terlampaui kuat (*overpowered*) di rentang fase awal pelatihan. Apabila Discriminator dapat membedakan data secara sempurna (probabilitas mendekati mutlak 1 atau 0), fungsi turunannya tidak lagi dapat memancarkan sinyal gradien (*gradient signal*) yang memadai ke Generator. Tanpa adanya laju aliran gradien balik ini, Generator akan terhenti kemampuannya untuk memperbarui bobot parameter dan gagal belajar.



Gambar 2.9: Diagram mekanistik terjadinya fenomena vanishing gradient akibat dominasi kekuatan Discriminator. Ketika Discriminator mencapai performa sempurna di awal pelatihan, kurva fungsi kerugiannya menjadi sangat landai (gradien mendekati nol). Kondisi ini memblokir aliran sinyal gradien balik (backpropagation) yang seharusnya dialirkan menuju Generator, menyebabkan pembaruan bobot parameter Generator terhenti total (stagnan).

2.7 EVOLUASI ARSITEKTUR MODERN EKSTENSI GAN

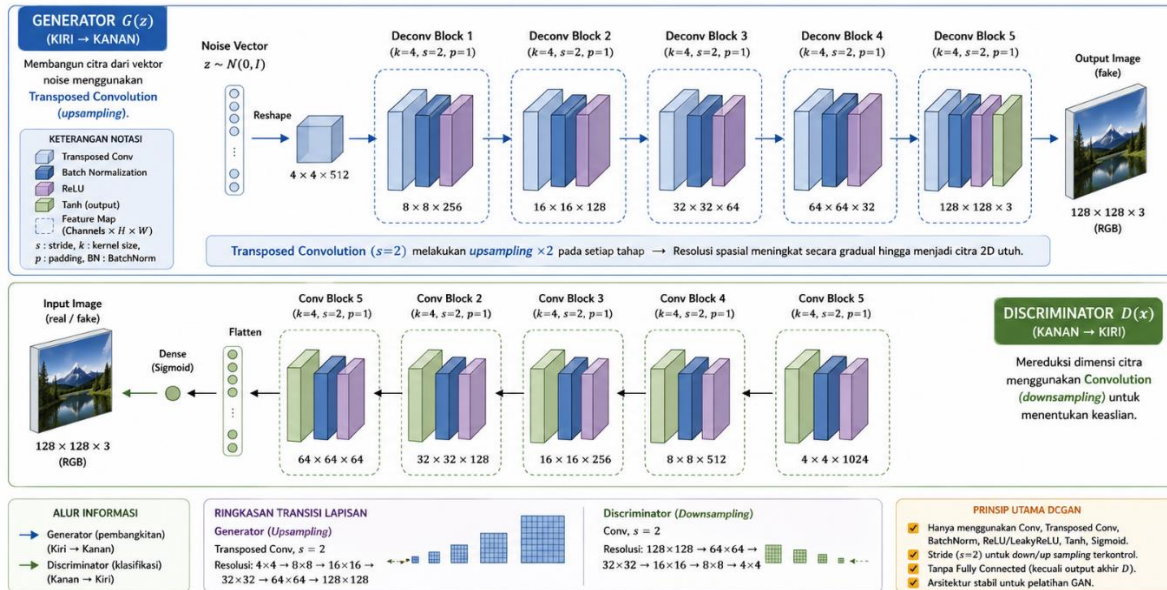
Untuk menstabilkan *loss*, mencegah *vanishing gradient*, dan meredam efek *mode collapse*, kerangka desain arsitektur GAN standar (*Vanilla GAN*) dikembangkan menjadi varian yang lebih mutakhir:

2.7.1. DEEP CONVOLUTIONAL GAN (DCGAN)

Arsitektur perintis yang merombak penggunaan lapisan tradisional *fully-connected layer* beralih menggunakan operator **konvolusi** (*Convolutional Layers*) untuk memproses citra dua dimensi dengan resolusi yang jauh lebih presisi.

- **Generator:** Mengadopsi metode **Transposed Convolution** (ConvTranspose2d) untuk melakukan pelebaran dimensi (*upsampling*) resolusi grid citra tanpa perlu menggunakan operator fitur reduksi seperti *pooling*.

- **Discriminator:** Menggunakan mekanisme reduksi resolusi (*downsampling*) murni melalui pendekatan manipulasi lompatan geseran matriks (*stride*) dari blok Conv2d.
- **Standarisasi Aktivasi:** Wajib mengimplementasikan **LeakyReLU** secara menyeluruh pada semua blok Discriminator guna menjaga sirkulasi rambatan sinyal minus gradien agar tidak mati (*dying ReLU*), serta penggunaan normalisasi **Batch Normalization** secara masif di kedua subsistem arsitektur.

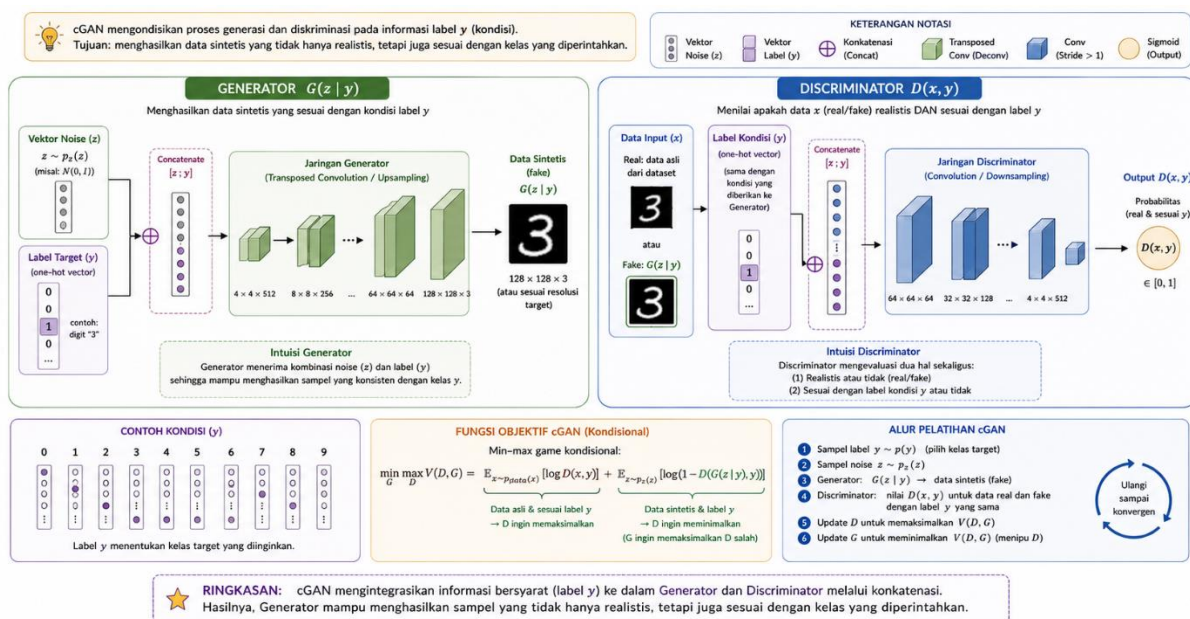


Gambar 2.10: Arsitektur struktural Deep Convolutional GAN (DCGAN) yang memperlihatkan transisi penggunaan lapisan konvolusi. Di sisi Generator (kiri ke kanan), vektor noise spasial dirangkai dan diperbesar secara gradual menggunakan Transposed Convolution (*upsampling*) hingga membentuk citra 2D utuh beresolusi tinggi. Di sisi Discriminator (kanan ke kiri), citra tersebut mengalami reduksi dimensi (*downsampling*) melalui operasi konvolusi dengan pengaturan stride untuk mengekstrak fitur penentu keaslian gambar.

2.7.2. CONDITIONAL GAN (CGAN)

Memodifikasi struktur ruang laten vektor *noise* standar (z) dengan turut menyertakan atribut kondisional atau label instruksi panduan kelas yang spesifik (direpresentasikan sebagai y). Melalui pendekatan cGAN ini, komputasi luaran dimensi Generator dikontrol menggunakan probabilitas bersyarat $G(z | y)$.

Arsitektur inovatif ini mendisrupsi tradisi proses penciptaan vektor acak buta dengan menyematkan kapabilitas untuk menargetkan luaran wujud gambar secara spesifik. Sebagai contoh praktisnya, pengguna dapat memberikan instruksi eksplisit agar Generator mengkonstruksi gambar angka '7', atau secara spesifik menghasilkan gambar wajah pria dewasa, yang dijamin sesuai dengan parameter kendali yang diinputkan.



Gambar 2.11: Diagram arsitektur Conditional GAN (cGAN) yang menunjukkan integrasi informasi bersyarat. Berbeda dengan GAN standar, jaringan Generator di sini menerima gabungan (concatenation) antara vektor noise acak (z) dan label target (y) untuk menghasilkan data sintetis spesifik $G(z | y)$. Di sisi lain, Discriminator juga menerima informasi label (y) yang sama sebagai pemandu untuk mengevaluasi apakah data yang diuji tidak hanya terlihat realistis, tetapi juga sesuai dengan kategori kelas yang diperintahkan.

PRAKTIKUM

3.1 DEPENDENSI

3.1.1. IMPORT LIBRARY

CODE

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

import torchvision
from torchvision import datasets, transforms

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
import time

# Cek versi PyTorch
print(f"PyTorch Version : {torch.__version__}")

# Cek ketersediaan GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device          : {device}")
```

3.1.2. MODUL PEMBUNGKUS UNTUK NN.SEQUENTIAL()

CODE

```
# Memastikan folder output ada
os.makedirs("output", exist_ok=True)
```

